Procesos de Desarrollo de Software

CLASE 10 – PATRÓN ADAPTER

Agenda de la clase

- Adapter
- Ejemplo simple: sensor de temperatura
- Aplicación en Magic Café

Adapter

Es un patrón **estructural**.

¿Qué hace?

- Permite que dos clases con interfaces incompatibles colaboren.
- El Adapter traduce el contrato que espera el cliente al contrato que ofrece una clase existente.

Se **sugiere** su utilización **cuando**:

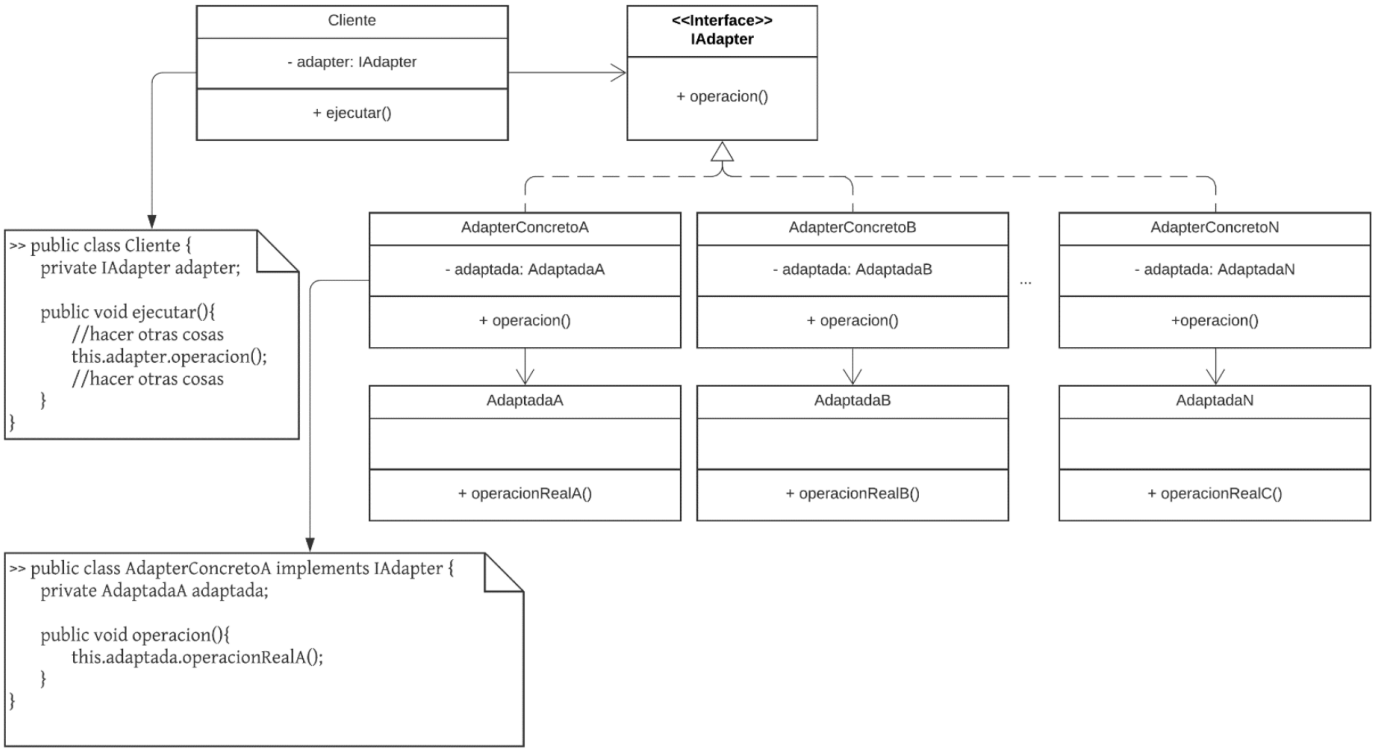
- Una biblioteca útil expone una API incompatible.
- Queremos aislar conversiones y detalles externos del resto de la aplicación.

Componentes

Componentes:

- **Interface Adapter**: define la firma del método que se utilizará como "puente" de acoplamiento a nuestro dominio.
- **Clases de Adapters concretos**: clases que implementan la interface Adapter, que se encargan de acoplar los componentes externos al dominio. Guardan referencia o hacen uso de las clases Adaptadas. Llaman a los métodos "reales" que resuelven el problema.
- **Clases Adaptadas**: clases externas al dominio, o no, que posee firmas incompatibles, o que todavía no conocemos (puede que todavía no estén creadas). Estas clases no se pueden/deben tocar.
- **Cliente**: clase que tiene referencia a la interface Adapter, cuyos objetos van a hacer uso de la funcionalidad que ésta les brinda.

Diagrama UML



El cliente conserva su contrato. El Adapter concentra la incompatibilidad.

Ejemplo

- Una app del clima necesita mostrar grados Celsius.
- Un sensor antiguo solo informa grados Fahrenheit.
- La app necesita una sola operación: leerCelsius().

Contrato esperado

leerCelsius()

Sensor existente

leerFahrenheit()

Solución sin Adapter

ClimaApp.java

```
SensorLegacy sensor = new SensorLegacy();

double fahrenheit =
sensor.leerFahrenheit();
double celsius = (fahrenheit - 32) * 5 /
9;

pantalla.mostrar(celsius);
```

Conversión repetida

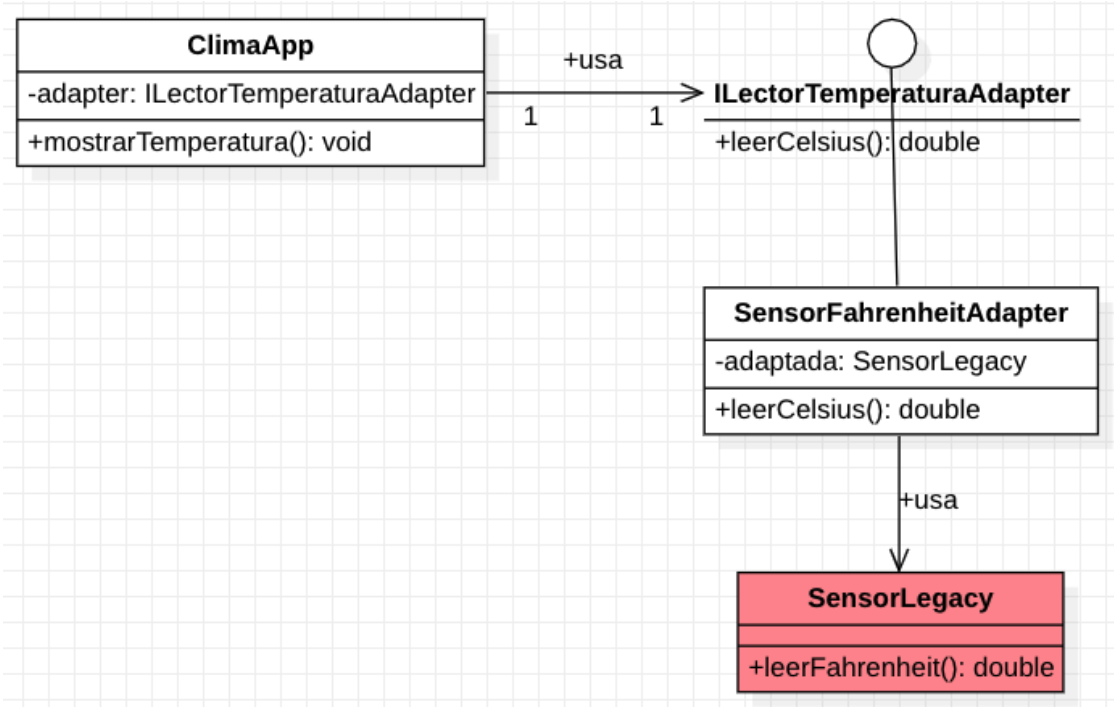
Cada cliente debe conocer la fórmula y la API legacy.

Acoplamiento

Cambiar el sensor obliga a modificar la aplicación.

La incompatibilidad queda distribuida en el código cliente.

Solución con Adapter



ClimaApp solo conoce LectorTemperatura.

Código del Adapter y uso

```java
public interface ILectorTemperaturaAdapter {
    double leerCelsius();
}

public class SensorFahrenheitAdapter implements ILectorTemperaturaAdapter {
    private SensorLegacy adaptada;

    public SensorFahrenheitAdapter(SensorLegacy adaptada) {
        this.adaptada = adaptada;
    }

    @Override
    public double leerCelsius() {
        double fahrenheit = adaptada.leerFahrenheit();
        return (fahrenheit - 32) * 5 / 9; // Convertir Fahrenheit a Celsius
    }
}
```

```java
// Uso
ILectorTemperaturaAdapter lector = new SensorFahrenheitAdapter(new SensorLegacy());
double temperaturaCelsius = lector.leerCelsius();
```

La conversión vive en un único objeto reemplazable.

Práctica

Realicemos el ejercicio de aplicación de patrones "**Módulo de envío de notificaciones**".

Magic Café ☕ ✨

- El proyecto confirma pedidos y calcula descuentos con Strategy.
- Ahora debe cobrar mediante una pasarela legacy que recibe centavos.
- Adapter permite integrar el proveedor sin contaminar el Service con esa API.

Client

MagicCafeService

Target

ProcesadorPago

Adapter + Adaptee

PasarelaPagoAdapter
PasarelaPagoLegacy

Problema de integración en Magic Café

Service acoplado a la pasarela

```
PasarelaPagoLegacy pasarela =
        new PasarelaPagoLegacy();

long centavos = Math.round(total * 100);
String comprobante =
        pasarela.ejecutarCobro(centavos);
```

El Service conoce la API externa

Debe invocar ejecutarCobro() y trabajar en centavos.

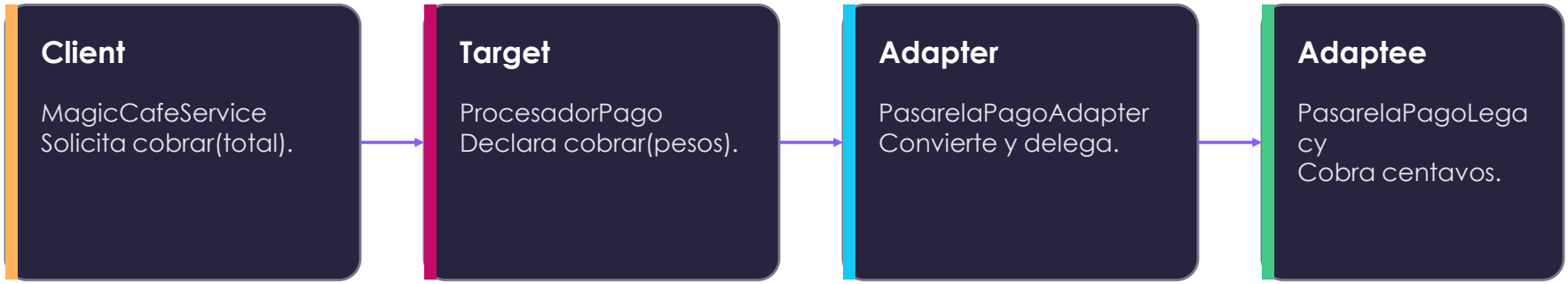
Cambiar proveedor modifica el caso de uso

La infraestructura externa queda mezclada con la lógica de aplicación.

La conversión puede duplicarse

Cada cobro podría repetir pesos × 100.

Roles de Adapter en Magic Café



- La aplicación define el contrato. Infraestructura resuelve la traducción.

Paso 1: definir el Target

ProcesadorPago.java

```
package com.uade.magiccafe.application.port;

public interface ProcesadorPago {

    String cobrar(double montoEnPesos);
}
```

Contrato de la aplicación

El caso de uso habla en pesos.

Puerto de salida

Aísla el proveedor detrás de una interfaz propia.

Application no importa clases de infrastructure.

Paso 2: implementar Adaptee y Adapter

PasarelaPagoLegacy.java

```
public class PasarelaPagoLegacy {

    public String ejecutarCobro(
            long montoEnCentavos) {
        return "LEG-...";
    }
}
```

PasarelaPagoAdapter.java

```
public class PasarelaPagoAdapter
        implements ProcesadorPago {

    private final PasarelaPagoLegacy legacy;

    public String cobrar(double pesos) {
        long centavos = Math.round(pesos * 100);
        return legacy.ejecutarCobro(centavos);
    }
}
```

- El Adapter implementa el Target y delega en la clase incompatible.

Paso 3: el Service usa el Target

MagicCafeService.java

```
private final ProcesadorPago procesadorPago;

public MagicCafeService(
        ProductoRepository productos,
        PedidoRepository pedidos,
        DescuentoRepository descuentos,
        ProcesadorPago procesadorPago) {
    this.procesadorPago = procesadorPago;
}

String comprobantePago =
        procesadorPago.cobrar(pedido.getTotal());
Pedido guardado = pedidoRepository.save(pedido);
```

Responsabilidad del Service

Solicita el cobro usando el contrato propio.

Responsabilidad del Adapter

Traduce pesos a centavos y llama a la pasarela.

El Composition Root conecta las piezas

Main.java

```
PasarelaPagoLegacy pasarelaLegacy =
        new PasarelaPagoLegacy();

ProcesadorPago procesadorPago =
        new PasarelaPagoAdapter(pasarelaLegacy);

MagicCafeService service = new MagicCafeService(
        productoRepository,
        pedidoRepository,
        descuentoRepository,
        procesadorPago
);
```

Infrastructure

Crea la pasarela y el Adapter.

Application

Recibe un ProcesadorPago listo para usar.

- La inyección manual mantiene visible la dependencia.

La traducción queda encapsulada

- MagicCafeService entrega el total en pesos.
- PasarelaPagoAdapter multiplica por 100 y redondea.
- PasarelaPagoLegacy recibe un long en centavos.
- El comprobante vuelve al DTO de confirmación.

Ejemplo: pedido de $7.700

Service	$7.700,00
Adapter	770.000
Legacy	770.000 cent.
DTO	LEG-A1B2C3D4

Flujo al confirmar y cobrar un pedido



- MagicCafeService nunca conoce el contrato de la pasarela legacy.

Gracias!

- ¿Preguntas?
- mailto: mstricagnoli@uade.edu.ar

